module ‘Cards’

Actions:

createDeck

creates a deck in the game (based on the deck from the database) with the same name

key: ‘createDeck’
payload:
required fields:
name: string; (unique name of deck)
set: string; (name of record consisting of the cards and their frequencies)
optional fields
public: boolean; (if ‘true’ - the deck is visible to all)
cardback: string;
label: string;
counter: boolean;
counterIncludesTopCard: boolean; (true by default. If set to false, the counter doesn’t count the top card)

facedown: boolean;
inspectDeck: boolean; (If ‘true’, a little eyeball icon appears in the bottom right corner of the deck if it’s being displayed. If anyone clicks on the eyeball icon, they see a “tooltip” that shows all the cards in the deck, in order.)
inspectors: string[]; (user Ids, who can inspect decks. If not set or empty, everyone can.)
enlargeOnHover: boolean; (If ‘true’, then when you hover over a deck, you get a tooltip showing a large image of the top card)
emptyImage: string; (image URL - for display that as the blank deck image in the central cards widget)
customName: string (deck name instead of the default)

createPublicDeck

creates an empty deck with ‘public_deck’ name and the central cards widget

key: ‘createPublicDeck’
payload:
required fields:
type: string; (widget type: ‘one_row’, ‘four_rows’, 'no_thanks', ‘square’)
optional fields
backgroundColor: string; (CSS value for widget)
textColor: string; (CSS value for widget)
borderColor: string; (CSS value for widget)
inspectDeck: boolean; (If ‘true’, a little eyeball icon appears in the bottom right corner of the deck if it’s being displayed. If anyone clicks on the eyeball icon, they see a “tooltip” that shows all the cards in the deck, in order.)
inspectors: string[]; (user Ids, who can inspect decks. If not set or empty, everyone can.)
enlargeOnHover: boolean; (If ‘true’, then when you hover over a deck, you get a tooltip showing a large image of the top card)

createPlayersDecks

creates empty decks with player id name

key: ‘createPlayersDecks’
payload:
required fields:
players: Array of strings; (user Ids)

createCustomDeck

creates empty deck

key: ‘createCustomDeck’
payload:
required fields:

name: string; (unique name of deck)
optional fields
public: boolean; (if 'true' - the deck is visible to all)
facedown: boolean; (default 'false')
cardback: string;
label: string;
counter: boolean;
counterIncludesTopCard: boolean; (true by default. If set to false, the counter doesn't count the top card)
inspectDeck: boolean; (If 'true', a little eyeball icon appears in the bottom right corner of the deck if it's being displayed. If anyone clicks on the eyeball icon, they see a "tooltip" that shows all the cards in the deck, in order.)
inspectors: string[]; (user Ids, who can inspect decks. If not set or empty, everyone can.)
enlargeOnHover: boolean; (If 'true', then when you hover over a deck, you get a tooltip showing a large image of the top card)
emptyImage: string; (image URL - for display that as the blank deck image in the central cards widget)
sorted: boolean; (true by default. If false disables sorting deck in inspect deck widget)

shuffleDeck

key: 'shuffleDeck'
payload:
required fields:
deck: string; (deck name)

dealDeck
moves cards from the deck to target hand(s)

key: 'dealDeck'
payload:
required fields:
targets: Array of strings OR string; (user id(s))
deck: string; (deck name)
optional fields:
qnt: number; (how many cards to deal to each target. If no value is set, the whole deck is dealt)
sortBy: string ('rank', 'points', 'hearts')
order: string ('asc', 'desc') default 'asc'
cardNames: string[]; (the names of the cards that will be dealt.)

playCards
moves cards from a hand to a deck

key: ‘playCards’
payload:
required fields:
actor: Array of one string OR string; (user id)
target: string; (deck name)
optional fields:
notification: string;
label: string; (result description template. E.g. ‘You passed $(cards) to $(target)’. Default: ‘You played $(cards)’)
minCards: number; (the minimum number of cards to be played on this turn, default 1)
maxCards: number; (the maximum number of cards that can be played this turn, default 1)
duration: number; (how long the action lasts in seconds. If unspecified, the action is untimed)
oneClick: boolean; (if true, then once maxCards are selected, the cards are played and the action is over; if false (which is the default), players have to click a Confirm button to actually finish the action and play the cards)
playable: string; (‘allAvailable’ (default), ‘availableCards’ - cards in the player's hand are allowed to be played through processing of the following parameters):
playableInclude: { cards?: string[]; - the names of the cards that will be available for playing.
suits?: string[] - the card suits that will be available for playing.
};
playableExclude: { cards?: string[]; - the names of the cards that will not be available for playing.
suits?: string[] - the card suits that will not be available for playing.
};
facedown: boolean;
overrideSpellCheck: boolean; whether skip word checking when resubmitting.
checkIfValid: boolean; whether to check a word for absence in the dictionary of real words.
checkIfInvalid: boolean; whether skip word checking when resubmitting.
privatePrompt: boolean; (false by default. If true show “Waiting for other players” at the top of the central widget for anyone who is not the actor)

returns result object: {
player: string - playerId,
target: string;
cards: Array of strings - played cards ids,
firstCardSuit: string,
}

<u>moveCards</u>
moves cards from deck(s) to a deck OR from hand(s) to a hand

key: ‘moveCards’
payload:
required fields:
type: string; ('hand' or 'deck');
from: Array of strings OR string;
to: string;
optional fields:

cardType: string OR array of strings; (A type(s) of card that is being moved. If not specified all cards pass the filter)
qnt: number; (how many cards to move from one source. If no value is set all cards move)
facedown: boolean;
cardNames: string[]; (the names of the cards that will be moved.)
reverseOrder: boolean (if 'true' cards are added to the beginning of the deck or hand)

recallCards

moves cards from hands(s) to a deck

key: 'recallCards'
payload:
required fields:
targets: Array of strings OR string; (user Ids)
deck: string; (unique name of deck to which the recalled cards get added)

optional fields:
type: string OR array of strings; (A type(s) of card that is being recalled. If not specified all cards pass the filter)
qnt: number; (A number of cards to be recalled per target. If **qnt** is not specified or exceeds the number of cards a target has of that type, recall all of them anyway)
facedown: boolean;
cardNames: string[]; (the names of the cards that will be moved.)

discard

key: 'discard'
payload:
required fields:
targets: Array of strings; (user Ids)
deck: string; (unique name of deck to which the discarded cards get added)
cards: Array of strings; (A list of card Ids. Discards the intersection of the cards in the hand and the list that is passed)
optional fields:
facedown: boolean;

showHand

key: 'showHand'
payload:
required fields:
from: Array of string OR string (user ids whose hand to show)
to: Array of string OR string**;** (user ids whom to show OR 'spectators' - for all spectators)

sortDeck

key: ‘sortDeck’
payload:
required fields:
deck: string; (unique name of deck)
sortBy: string (‘rank’, ‘points’ - the card object field)
optional fields:
order: string (‘asc’, ‘desc’)

takeFivePlaceCards

a special action for the ‘Take 5’ game to distribute cards in the central widget by rows

key: ‘takeFivePlaceCards’
payload:
optional fields:
takeRowDuration: number (default 30 sec.)

setCardsPile

key: ‘setCardsPile’
payload:
required fields:
name: string; (the “name” field of the card you wish to pile)
optional fields:
style: 'left' | 'middle' | 'right' (where in the card to display xN text. Default is “middle”)
color: string (a hex or css for the color of the xN text. Default ‘black’)
fontSize: number (how big should the text be as a percentage of the card. Default is 10%)

createGenericCardWidget

(widget ID: ‘GenericCardWidget’)

key: ‘createGenericCardWidget’
payload:
required fields:
dimensions: [number, number];
decks: string[];
cardback: string;
optional fields:
ratio: string; (default ‘0.625’)
backgroundColor: string; (CSS value for widget)
textColor: string; (CSS value for widget)

borderColor: string; (CSS value for widget)
inspectDeck: boolean; (If 'true', a little eyeball icon appears in the bottom right corner of the deck if it's being displayed. If anyone clicks on the eyeball icon, they see a "tooltip" that shows all the cards in the deck, in order. It applies to all decks in the widget.)
inspectors: string[]; (user Ids, who can inspect decks. If not set or empty, everyone can.)
enlargeOnHover: boolean; (If 'true', then when you hover over a deck, you get a tooltip showing a large image of the top card. It applies to all decks in the widget.)
backgroundImage: string; (image URL)

createVideoBoxDecks

creates empty decks with name "videobox_playerId"

key: 'createVideoboxDecks'
payload:
required fields:
players: Array of strings; (user Ids)
optional fields:
facedown: boolean; (default 'false')
cardback: string;

noThanksPlaceCards

a special action for the 'No Thanks' game to distribute cards in the central widget

key: 'noThanksPlaceCards'
payload:
actors: Array of strings; (user ids)
startActor: string; (user id)

return result: {
isCardsTaken: boolean,
lastCardOwner: string
}

selectCentralWidgetDeck

He added a list of decks that will be interactive.

key: 'selectCentralWidgetDeck'
payload:
required fields:
actors: string OR Array of strings; (user ids)

optional fields:

terminationCondition: get_needed_votes (terminates the action if 'neededVotes' players make a selection)
neededVotes: the number of votes needed to terminate the action
decks : array of string (clickable decks ids, if undefined all decks are clickable)
duration: number;
question: string;
defaultSelect: string; (deck id)
privatePrompt: boolean; (false by default. If true show "Waiting for other players" at the top of the central widget for anyone who is not the actor)
pulse: boolean; (pulse animation)

return result as Record: {
userId: deckId,
…
}

flipOverTopCard

key: 'flipOverTopCard'
payload:
required fields:
deck: string; (unique name of deck)

optional fields:
delay: number; (action delay after the card is turned over, in seconds)

createCard

creates a new card based on a blank image and text. The height of the source image is 600 pixels. The width is specified by the **ratio** parameter. The image is saved in Cloudinary service in the game_created_cards folder.

key: 'createCard'
payload:
required fields:
deck: string; (id of the deck in which the new card will be placed)

optional fields:
cardText : string; (will be saved in the game state as card 'text' parameter)
fontHeightPercentage: number; (sets the font size as a percentage of the image height. The default font size is 40px)
ratio: number; (default '0.625')
cardImage: string; (image URL on top of which the cards will be built)
background: string; (a hex color for the background of the card (if no image is provided))
textColor: string; (a hex color for the text. Default: '#2b03a3')

card parameters that will be saved in the game state:

name: string;**,**
type: string;**,**
label: string;**,**
facedown: boolean;
description: string;**,**
rank: number;**,**
points: number;**,**
weight: number;**,**
suit: string;**,**
enlargeOnHover: boolean;

createTextCards

creates multiple cards in the same way as the createCard action.

key: ‘createTextCards’’
payload:
required fields:
deck: string; (id of the deck in which the new card will be placed)
cardText : string[];

optional fields:
fontHeightPercentage: number; (sets the font size as a percentage of the image height. The default font size is 40px)
ratio: number; (default ‘0.625’)
cardImage: string; (image URL on top of which the cards will be built)
background: string; (a hex color for the background of the card (if no image is provided))
textColor: string; (a hex color for the text. Default: '#2b03a3')

card parameters that will be saved in the game state:
name: string;**,**
type: string;**,**
label: string;**,**
facedown: boolean;**,**
description: string;**,**
rank: number;**,**
points: number;**,**
weight: number;**,**
suit: string;**,**
enlargeOnHover: boolean;

setDeckInspectors

key: ‘setDeckInspectors’
payload:
required fields:
deck: string OR array of strings; (unique name of deck)
inspectors: array of strings (user ids)

setInspectDeck

key: ‘setInspectDeck’
payload:
  required fields:
    deck: string OR array of strings; (unique name of deck)
    inspectDeck: boolean;

setDeckLabels

key: ‘setDeckLabels’
payload:
  required fields:
    decks: array of strings; (unique name of deck)
    labels: array of strings;

setDeckLabel

key: ‘setDeckLabel’
payload:
  required fields:
    deck: string; (unique name of deck)
    label: string;

highlightDecks

key: ‘highlightDecks’
payload:
  required fields:
    decks: array of strings; (unique name of deck)
    color: string;

removeHighlightDecks

key: ‘removeHighlightDecks’
payload:
  required fields:
    decks: array of strings; (unique name of deck)

removeAllHighlightDecks

key: ‘removeAllHighlightDecks’